4 Soluçao

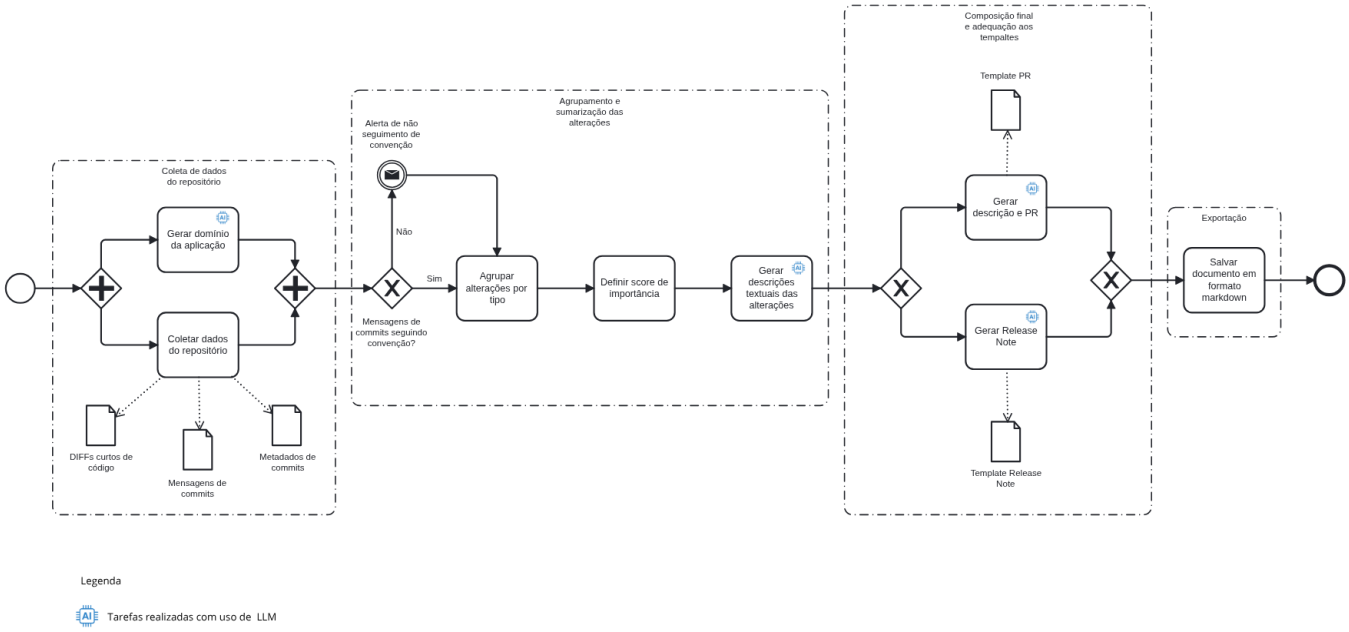
4.1 Visão geral

A solução proposta neste trabalho consiste em uma ferramenta que automatiza a redação de descrições de *pull requests* e de *release notes* a partir de informações já disponíveis no repositório de código. Em termos gerais, a ferramenta opera como um *pipeline* de processamento que consome artefatos produzidos no fluxo cotidiano de desenvolvimento, aplica regras de engenharia de software combinadas a modelos de linguagem leves e privados e devolve textos estruturados, prontos para uso em processos de revisão de código e gestão de releases. O diagrama BPMN apresentado nesta seção sintetiza esse fluxo de ponta a ponta, desde a coleta de dados até a exportação dos documentos gerados.

De forma ampla, o *pipeline* é organizado em três macrofases. Na primeira, de coleta de dados do repositório, a ferramenta inspeciona o repositório local do desenvolvedor, extrai diffs curtos de código, mensagens de *commit* e metadados associados, e, em paralelo, infere o domínio da aplicação a partir de arquivos documentais presentes no projeto. Em seguida, na fase de agrupamento e sumarização das alterações, os *commits* são organizados por tipo de mudança com base em convenções semânticas de mensagens; quando essas convenções não são seguidas, a ferramenta emite um alerta ao usuário e trata tais registros de forma diferenciada. Ainda nessa fase, é calculado um escore de importância para cada alteração e são produzidas descrições textuais sintéticas das mudanças, utilizando um modelo de linguagem configurado para operar com contexto limitado e em ambiente controlado.

Na terceira macrofase, de composição final e exportação, as descrições das alterações são inseridas em modelos textuais previamente definidos para descrições de *pull requests* ou para *release notes*. A ferramenta completa os demais campos dos templates com apoio do modelo de linguagem, adequando o tom, a estrutura e o nível de detalhe ao tipo de artefato desejado. Ao final do fluxo, o resultado é salvo em formato *markdown*, facilitando sua incorporação em ferramentas de hospedagem de código e em *pipelines* de integração e entrega contínua. Assim, a visão geral da solução é a de um mecanismo intermediário entre o histórico de desenvolvimento e os artefatos de comunicação de mudanças, desenhado para reduzir esforço manual, reforçar o uso de boas práticas de versionamento e produzir documentação mais clara e consistente, sem expor o repositório a serviços externos.

Figura 1: Diagrama BPMN que descreve processo do pipeline



Fonte: Do Autor

4.2 Requisitos do sistema

Os requisitos do sistema proposto são organizados em duas categorias principais: requisitos funcionais e requisitos não funcionais. Os requisitos funcionais descrevem os serviços e comportamentos que a ferramenta deve oferecer para cumprir seu objetivo central de extrair informações de repositórios de código, processar commits e apoiar a geração automática de descrições de *pull requests* e notas de versão. Já os requisitos não funcionais especificam restrições e propriedades de qualidade associadas a esses serviços, como desempenho, privacidade, usabilidade, portabilidade e manutenibilidade, assegurando que o sistema não apenas realize as funções desejadas, mas o faça de maneira eficiente, segura e adequada ao contexto de uso.

A seguir apresenta-se a tabela de documentação dos Requisitos Funcionais.

Tabela 2: Requisitos Funcionais do Sistema

ID	**Nome**	**Descrição**
RF01	Conectar-se a um repositório Git local	O sistema deve permitir configurar o caminho de um repositório Git local, a partir do qual todas as demais etapas do pipeline serão executadas.

RF02	Extrair histórico de commits	O sistema deve extrair commits de um intervalo configurável (por exemplo, entre duas tags, entre dois hashes, por branch, por data ou por número de versão).
RF03	Coletar metadados de cada commit	Para cada commit selecionado, o sistema deve obter, no mínimo: hash; autor e e-mail; data e hora; mensagem completa do commit; lista de arquivos modificados; número de linhas adicionadas/removidas; diff.
RF04	Identificar arquivos "documentais"no repositório	O sistema deve varrer o repositório e identificar arquivos considerados documentais, com base em extensões configuráveis (por exemplo: .md, .txt, .rst, .docx, .pdf extraível, .yaml, .yml, .json, etc.).
RF05	Construir contexto de domínio da aplicação	O sistema deve: a) concatenar ou organizar o conteúdo desses arquivos documentais; b) enviar esse conteúdo para um modelo de linguagem; c) solicitar que o modelo preencha um template estruturado (XML) contendo campos como: nome e breve descrição do sistema; domínio de negócio; principais funcionalidades; principais perfis de usuários; tecnologias e frameworks principais; restrições relevantes.
RF06	Armazenar o modelo de domínio	O sistema deve persistir o resultado do preenchimento de domínio (por exemplo, um arquivo XML/JSON em diretório do projeto), de forma que possa ser reutilizado em execuções futuras sem recomputar tudo.
RF07	Classificar commits por tipo de alteração (taxonomia)	O sistema deve classificar cada commit em categorias de tipo de alteração, a partir da mensagem de commit, usando uma taxonomia baseada em convenções semânticas configurável (por exemplo, feat, fix, chore, docs, refactor, test, perf, build, ci, style, etc.).

RB01	Mensagens fora da convenção	Se a mensagem de commit não contiver nenhum identificador de tipo reconhecido na taxonomia, o sistema deve: classificar o commit como "outros"(ou categoria equivalente); e registrar um alerta indicando que a mensagem não segue a convenção, recomendando o uso correto para pleno funcionamento da ferramenta.
RF08	Agrupar commits por tipo de alteração	O sistema deve agrupar os commits por tipo de alteração, de acordo com a taxonomia, para uso posterior na geração de descrições e na organização de *release notes*.
RF09	Calcular um score de importância por commit	O sistema deve calcular um score de importância para cada commit, usando heurísticas baseadas em metadados, tais como: tamanho do diff (linhas adicionadas/removidas); número de arquivos afetados; presença de palavras-chave na mensagem; arquivos de áreas críticas.
RF10	Classificar importância em faixas	O sistema deve mapear o score numérico de importância para faixas qualitativas (por exemplo, baixa, média, alta, crítica) para facilitar apresentação ao usuário.
RF11	Ordenar commits por importância	Ao exibir ou preparar listas de alterações, o sistema deve ser capaz de ordenar commits dentro de cada grupo pelo score de importância.
RF13	Garantir que o resumo seja factualmente ancorado	O sistema deve instruir o modelo de linguagem para que o resumo não inclua informações não presentes no contexto (redução de alucinação) e deve sinalizar ao usuário se ocorrerem respostas fora do padrão esperado.
RF14	Armazenar descrições de alterações	O sistema deve armazenar, para cada commit: tipo de alteração, score e faixa de importância, resumo textual e demais metadados para uso posterior na composição de PRs e *release notes*.

RF15	Carregar e gerenciar templates de descrições	O sistema deve permitir configurar templates distintos para: descrição de Pull Request; *Release Notes* de uma versão. Esses templates devem prever campos como título, resumo geral, seções por tipo de alteração, avisos de migração, problemas conhecidos, riscos, entre outros.
RF17	Ajustar o contexto para *Release Notes* usando o domínio da aplicação	Ao gerar uma *release notes*, o sistema deve incorporar o modelo de domínio (RF05 e RF06) ao contexto de chamada da LLM, para que a redação seja alinhada ao vocabulário e ao público daquele sistema específico.
RF18	Completar campos de alto nível via LLM	O sistema deve usar o modelo de linguagem para completar campos mais abstratos dos templates, como: resumo executivo da versão; texto introdutório para usuários finais; notas para equipes internas. Sempre com instruções para respeitar o conteúdo das descrições já geradas e não inventar funcionalidades inexistentes.
RF20	Exportar resultados em formatos compatíveis com plataformas de hospedagem de código	O sistema deve exportar as descrições geradas em formato Markdown.
RF23	Relatar conformidade com convenções de mensagens	O sistema deve produzir um relatório simples (por exemplo, em texto ou Markdown) com estatísticas sobre aderência às convenções de mensagens: porcentagem de commits corretamente classificados; commits em "outros"; exemplos de boas e más práticas.

RF24	Configuração de parâmetros do pipeline	O sistema deve permitir ajustar, via arquivo de configuração ou CLI: taxonomia de tipos de commits; extensões consideradas documentais; pesos usados no cálculo do score de importância; limites de tamanho de diff para envio ao modelo; idioma de saída; templates de PR e *release notes*; modelo(s) de linguagem a serem utilizados.

Fonte: Do Autor

A seguir apresenta-se a tabela de documentação dos Requisitos Não-Funcionais.

Tabela 3: Requisitos Não Funcionais do Sistema

ID	**Nome**	**Descrição**
RNF01	Privacidade e confidencialidade	O sistema deve permitir execução em ambiente totalmente local ou em infraestrutura privada, evitando envio de código-fonte ou artefatos sensíveis a serviços externos, exceto se explicitamente configurado pelo usuário.
RNF02	Uso de modelos de linguagem leves	O sistema deve privilegiar modelos de linguagem leves (por exemplo, modelos menores rodando localmente ou em contêiner dedicado), de forma que: seja viável executar em máquinas de desenvolvedores ou servidores internos com recursos moderados; reduza custos de execução contínua.
RNF03	Desempenho	O sistema deve ser capaz de processar um conjunto típico de commits de uma release (por exemplo, algumas dezenas) em tempo adequado ao fluxo de trabalho de desenvolvimento, sem introduzir atrasos significativos no ciclo de PR ou de release.

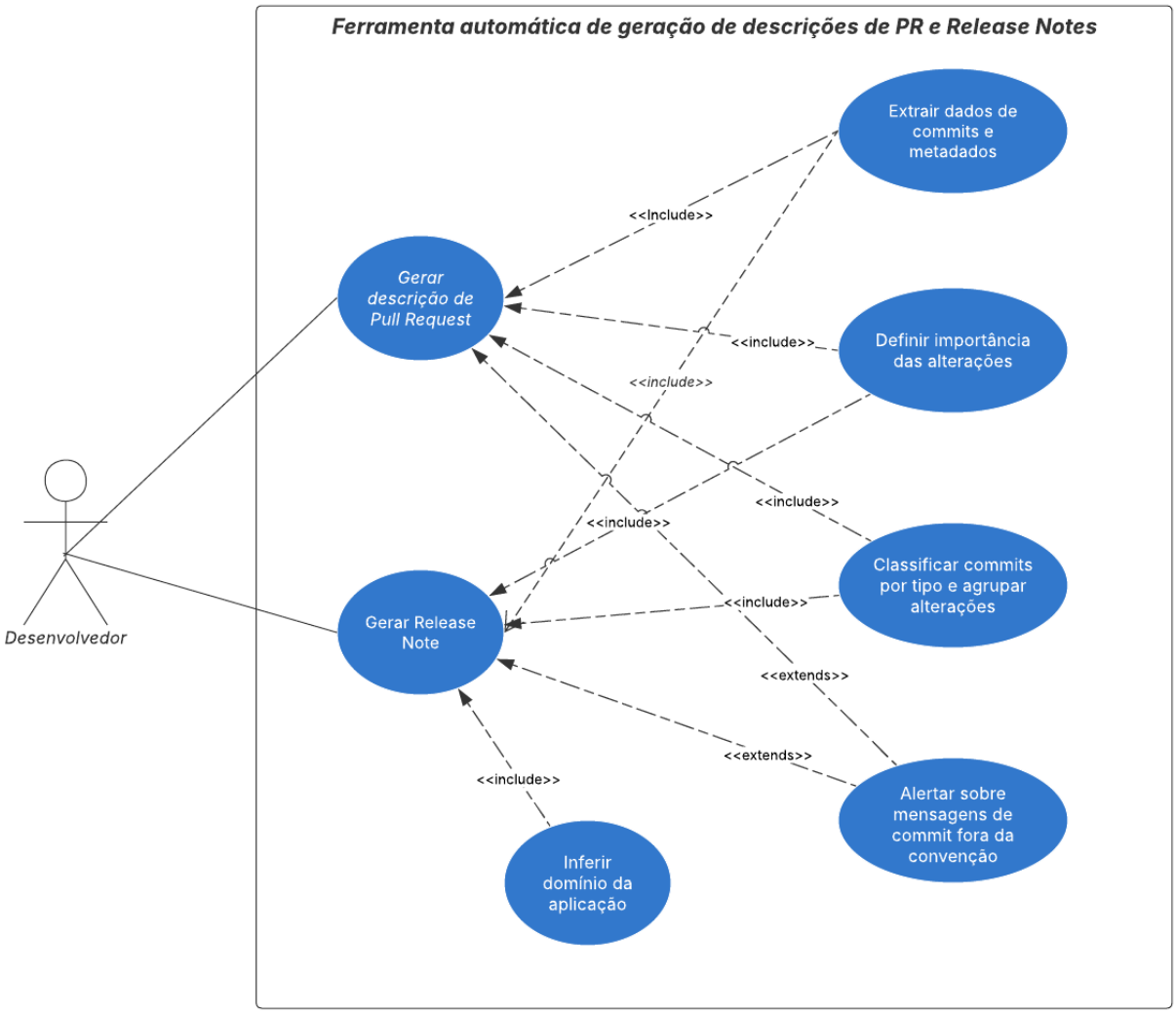
RNF04	Robustez e tolerância a falhas	Se alguma etapa falhar (por exemplo, LLM indisponível, arquivo de documentação inválido), o sistema deve: registrar o erro em log; informar o usuário de forma clara; sempre que possível, continuar o processamento parcial em vez de abortar silenciosamente.
RNF05	Usabilidade	O sistema deve: oferecer comandos e mensagens de ajuda claros na CLI; apresentar mensagens de erro descritivas; utilizar textos de descrição em linguagem simples, mesmo quando o uso é técnico.
' RNF07	Portabilidade	O sistema deve disponibilizar uma imagem Docker que encapsule todas as dependências.
RNF08	Segurança	O sistema não deve: registrar em log trechos completos de código sensível sem consentimento; expor credenciais, tokens de acesso ou dados sensíveis em arquivos de configuração; criar endpoints de rede desnecessários em modo local. Quando usar APIs externas, deve seguir boas práticas de armazenamento de segredos.
RNF10	Internacionalização	O sistema deve permitir configurar o idioma-alvo das descrições (por exemplo, português e inglês) e adaptar templates conforme o idioma escolhido.

Fonte: Do Autor

4.3 Casos de uso

Os casos de uso apresentados a seguir descrevem, em nível de interação entre atores e sistema, as principais funcionalidades da ferramenta de geração automática de descrições de *Pull Requests* e *release notes*. Dessa forma, o diagrama de casos de uso contribui para consolidar uma visão compartilhada do escopo funcional do sistema, apoiando a comunicação em conformidade com práticas recomendadas de modelagem de requisitos na Engenharia de Software e no uso de UML

Figura 2: Diagrama de Casos de Uso



Fonte: Do Autor